📘 PROJECT EXECUTION DOCUMENT

Network Intrusion Detection using NSL-KDD

1️⃣ Operating System

✅ Recommended: Ubuntu (Linux)

Why?

- Better ML compatibility
- Smooth TensorFlow / PyTorch setup
- Easier GPU support
- Real-world deployment environment
- Better for cybersecurity context

If not available:

- Use WSL2 (Windows Subsystem for Linux)

2️⃣ Required Software Installation

Step 1: Install System Tools

```
sudo apt update
sudo apt upgrade
sudo apt install build-essential curl git
```

Step 2: Install Python

Recommended version:

> Python 3.10 or 3.11

Check:

```
python3 --version
```

If not installed:

```
sudo apt install python3 python3-pip python3-venv
```

3️⃣ Should We Use Virtual Environment?

✅ YES — Mandatory

Why?

- Avoid dependency conflicts
- Professional practice
- Clean project structure
- Easy to deploy

Create Virtual Environment

Inside project folder:

```
python3 -m venv venv
source venv/bin/activate
```

Now install all libraries inside this environment.

4️⃣ Required Python Libraries

Install all at once:

```
pip install numpy pandas matplotlib seaborn scikit-learn
pip install imbalanced-learn
pip install xgboost lightgbm
pip install tensorflow
pip install shap
pip install joblib
pip install jupyter
pip install streamlit
```

Optional (advanced):

```
pip install optuna
pip install keras-tuner
```

---

5 Recommended Project Folder Structure

```
IntrusionDetectionProject/
│
├── data/
│   ├── KDDTrain+.txt
│   ├── KDDTest+.txt
│
├── notebooks/
│   ├── 01_EDA.ipynb
│   ├── 02_FeatureEngineering.ipynb
│   ├── 03_ML_Models.ipynb
│   ├── 04_CNN_Model.ipynb
│
├── src/
│   ├── preprocessing.py
│   ├── feature_selection.py
│   ├── models.py
│   ├── evaluation.py
│
├── app/
│   ├── streamlit_app.py
│
├── saved_models/
│
├── requirements.txt
│
└── README.md
```

This structure makes your project look professional.

---

6 From Where Should We Start?

We start in this order:

1. Dataset Understanding
2. Data Preprocessing
3. Feature Engineering
4. Baseline ML Models
5. Class Imbalance Handling
6. Advanced Models (XGBoost)
7. CNN
8. Hybrid Model
9. Visualization & Deployment
10. Documentation

📅 30-Day Execution Plan

🗓️ WEEK 1 – Foundation & Data Understanding

Day 1

- Setup Ubuntu
- Setup Python
- Create virtual environment
- Install libraries
- Setup folder structure

Day 2

- Load dataset
- Understand columns
- Add column names
- Identify categorical features

Day 3

- Perform EDA:
    - Distribution of attacks
    - Protocol types
    - Services

    - Flag analysis

Day 4

- Handle categorical variables (One-hot encoding)
- Convert labels (Binary & Multi-class versions)

Day 5

- Apply scaling (MinMaxScaler)
- Split train/test properly

Day 6

- Visualize correlations
- Heatmap analysis

Day 7

- Clean code and modularize preprocessing

🗓️ WEEK 2 – Feature Engineering + ML Models

Day 8

- Feature importance using ExtraTrees

Day 9

- Remove low-importance features
- Try PCA

Day 10

- Train:
    - Logistic Regression
    - Decision Tree

Day 11

- Train:
    - Random Forest
    - SVM

Day 12

- Train:
    - KNN
    - AdaBoost

Day 13

- Compare:
    - Accuracy
    - Precision
    - Recall
    - F1-score
    - ROC-AUC

Day 14

- Write baseline comparison table

Now you already outperform 70% of students.

🗓️ WEEK 3 – Make It Advanced

Day 15

- Implement SMOTE (class imbalance handling)

Day 16

- Retrain best models with SMOTE
- Compare improvements

Day 17

- Implement XGBoost
- Hyperparameter tuning

Day 18

- Try LightGBM

Day 19

- Hyperparameter tuning (GridSearch / Optuna)

Day 20

- SHAP analysis (Explainability)

This step makes your project stand out.

Day 21

- Save best ML model using joblib

🗓️ WEEK 4 – Deep Learning & Hybrid

Day 22

- Prepare data for CNN
- Reshape features

Day 23

- Build simple CNN

Day 24

- Improve CNN (dropout, batchnorm)

Day 25

- Compare CNN vs RF vs XGBoost

Day 26

- Build Hybrid concept:
    - Use RF selected features
    - Train CNN on reduced feature set

Day 27

- Generate:
    - Confusion matrix
    - ROC curves
    - Feature importance plots

Day 28

- Build Streamlit Dashboard

Day 29

- Write professional README
- Document:
    - Methodology
    - Results
    - Improvements

Day 30

- Final polish
- PPT preparation
- Viva preparation

🚀 How To Make Your Project Better Than Others

Most students will:

❌ Train RandomForest
❌ Show accuracy
❌ Stop there

You will:

✅ Handle class imbalance
✅ Do feature selection
✅ Compare multiple models
✅ Use explainability (SHAP)
✅ Build CNN
✅ Try hybrid model
✅ Create dashboard
✅ Save model
✅ Structure project professionally

That makes your project industry-grade.

🏆 Extra Improvements (Optional)

If you want elite-level:

- Deploy model using Flask API
- Dockerize project
- Try real-time packet simulation
- Use transfer learning
- Use Optuna for tuning
- Add training time comparison

🧠 Interview Preparation Angle

You must be able to explain:

- Why SMOTE?
- Why feature selection?
- Why RandomForest works well?
- What is ROC-AUC?
- Why U2R detection is difficult?
- What is class imbalance?
- Why hybrid model improves detection?

🎯 Final Tech Stack Summary

Component	Tool
OS	Ubuntu
Language	Python 3.10
ML	scikit-learn
DL	TensorFlow
Advanced ML	XGBoost
Visualization	Seaborn, Matplotlib
Explainability	SHAP
Deployment	Streamlit
Model Saving	joblib

📦1️⃣ requirements.txt

Create a file named:

requirements.txt

Paste this inside:

```
# Core Libraries
numpy==1.26.4
pandas==2.2.2
matplotlib==3.8.4
seaborn==0.13.2

# Machine Learning
scikit-learn==1.4.2
imbalanced-learn==0.12.2
xgboost==2.0.3
lightgbm==4.3.0

# Deep Learning
tensorflow==2.16.1

# Explainability
shap==0.45.1

# Hyperparameter Tuning
optuna==3.6.1

# Utilities
joblib==1.4.2
jupyter==1.0.0

# Deployment
streamlit==1.35.0
```

Install Everything

After activating virtual environment:

```
pip install -r requirements.txt
```

🏗️2️⃣ Professional Starter Code Template

We follow this structure:

```
IntrusionDetectionProject/
│
├── data/
├── src/
│   ├── preprocessing.py
│   ├── feature_selection.py
│   ├── models.py
│   ├── cnn_model.py
│   ├── evaluation.py
│
├── app/
│   └── streamlit_app.py
│
├── main.py
├── requirements.txt
└── README.md
```

🧠 3️⃣ Starter Code Files

🔹 src/preprocessing.py

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split


def load_data(path):
    df = pd.read_csv(path, header=None)
    return df


def encode_categorical(df, categorical_columns):
    df = pd.get_dummies(df, columns=categorical_columns)
    return df


def scale_features(X):
    scaler = MinMaxScaler()
    X_scaled = scaler.fit_transform(X)
    return X_scaled
```

```
def split_data(X, y, test_size=0.2):
    return train_test_split(X, y, test_size=test_size, random_state=42)
```

🔹 src/feature_selection.py

```
from sklearn.ensemble import ExtraTreesClassifier
import pandas as pd


def select_features(X, y, top_n=20):
    model = ExtraTreesClassifier(n_estimators=100)
    model.fit(X, y)

    feature_importance = pd.Series(
        model.feature_importances_,
        index=X.columns
    )

    top_features = feature_importance.nlargest(top_n).index
    return X[top_features]
```

🔹 src/models.py

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


def train_random_forest(X_train, y_train):
    model = RandomForestClassifier(n_estimators=200, max_depth=15)
    model.fit(X_train, y_train)
    return model


def train_svm(X_train, y_train):
    model = SVC(probability=True)
    model.fit(X_train, y_train)
    return model
```

```
def train_logistic(X_train, y_train):
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)
    return model


def evaluate_model(model, X_test, y_test):
    preds = model.predict(X_test)
    return accuracy_score(y_test, preds)
```

🔹 src/cnn_model.py

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv1D, Flatten, MaxPooling1D, Dropout


def build_cnn(input_shape, num_classes):
    model = Sequential()

    model.add(Conv1D(64, 3, activation='relu', input_shape=input_shape))
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.3))

    model.add(Flatten())
    model.add(Dense(64, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

🔹 src/evaluation.py

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
```

```
import seaborn as sns


def print_classification_report(y_true, y_pred):
    print(classification_report(y_true, y_pred))


def plot_confusion_matrix(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d')
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
```

🔹 main.py (Project Entry Point)

```
from src.preprocessing import load_data, encode_categorical, scale_features, split_data
from src.feature_selection import select_features
from src.models import train_random_forest, evaluate_model

import pandas as pd


def main():

    # Load dataset
    df = load_data("data/KDDTrain+.txt")

    # Example: Define categorical columns
    categorical_columns = [1, 2, 3]

    # Encoding
    df = encode_categorical(df, categorical_columns)

    # Separate features and label
    X = df.iloc[:, :-1]
    y = df.iloc[:, -1]

    # Feature Selection
    X = select_features(X, y)

    # Scaling
    X = scale_features(X)
```

```
    # Split
    X_train, X_test, y_train, y_test = split_data(X, y)

    # Train Model
    model = train_random_forest(X_train, y_train)

    # Evaluate
    acc = evaluate_model(model, X_test, y_test)

    print(f"Model Accuracy: {acc}")


if __name__ == "__main__":
    main()
```

🔹 app/streamlit_app.py (Optional Dashboard)

```
import streamlit as st
import joblib
import numpy as np

st.title("Network Intrusion Detection System")

model = joblib.load("../saved_models/rf_model.pkl")

input_data = st.text_input("Enter feature values separated by commas")

if st.button("Predict"):
    values = np.array([float(x) for x in input_data.split(",")]).reshape(1, -1)
    prediction = model.predict(values)
    st.write("Prediction:", prediction)
```

Run dashboard:

```
streamlit run app/streamlit_app.py
```